Lostify

A Cloud-Native, Microservice-Based Lost & Found Platform `v2.0`

Zain Afzal · Babar Ali · Muhammad Hamza Azeem
Team 04

Serverless Cloud Application Automation
Prof. Sashko Ristov · July 1, 2026

`github.com/Babarali2k21/Lostify`
`Live: 44.220.148.111:3001`

INTRODUCTION

Problem & Motivation

Traditional Lost & Found

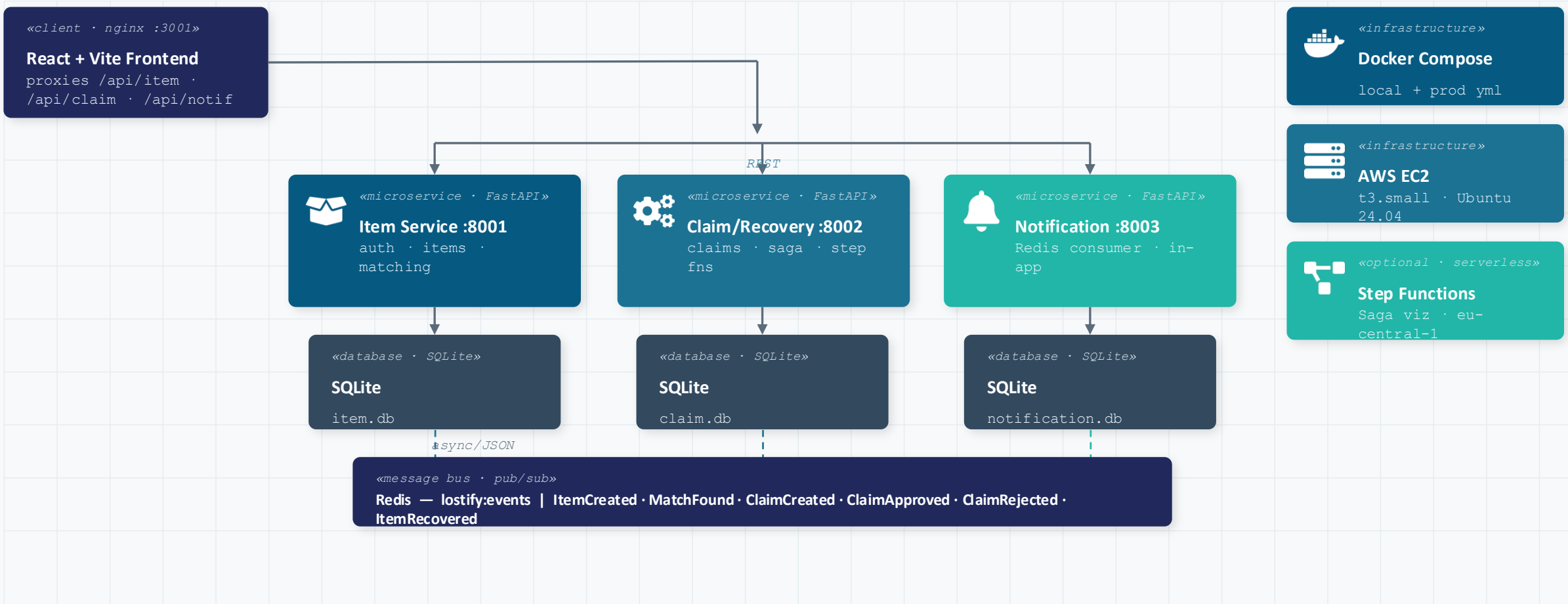
- ✗ Calling an office or visiting a physical desk
- ✗ Asking in chat groups, hoping for a post
- ✗ No way to track if a match already exists
- ✗ No record of whether an owner was notified
- ✗ Handover completion is never confirmed

Lostify

- ✓ Structured lost / found reports (title, description, item type)
- ✓ Automatic keyword-overlap matching of lost & found reports
- ✓ Owner notified in-app when a match is found
- ✓ Claim workflow — user submits, finder approves or rejects
- ✓ Full lifecycle tracked end-to-end with Saga compensation

SOLUTION

Overall Architecture — Container Diagram



Microservice Architecture — Why Three Services?

Service	Responsibility	Database
`Item Service :8001`	Auth (register/login/JWT), lost/found reports, keyword matching, item states (OPEN→MATCHED→RESERVED→RECOVERED)	SQLite (item.db)
`Claim/Recovery :8002`	Claims, claim states (PENDING→APPROVED|REJECTED), Saga orchestration, compensation, AWS Step Functions sync	SQLite (claim.db)
`Notification :8003`	Redis pub/sub consumer, in-app notification log, eventId deduplication	SQLite (notification.db)

Key service boundary decisions:

- No separate User Service — auth is a cross-cutting concern embedded in Item Service per course guidance
- Claim/Recovery is separate from Item Service — claims have their own states and Saga compensation rules
- Notification is separate — event processing can lag or fail independently of item and claim logic

Advantages

- Independent deployment
- Fault isolation
- Scalability per service
- Maintainability

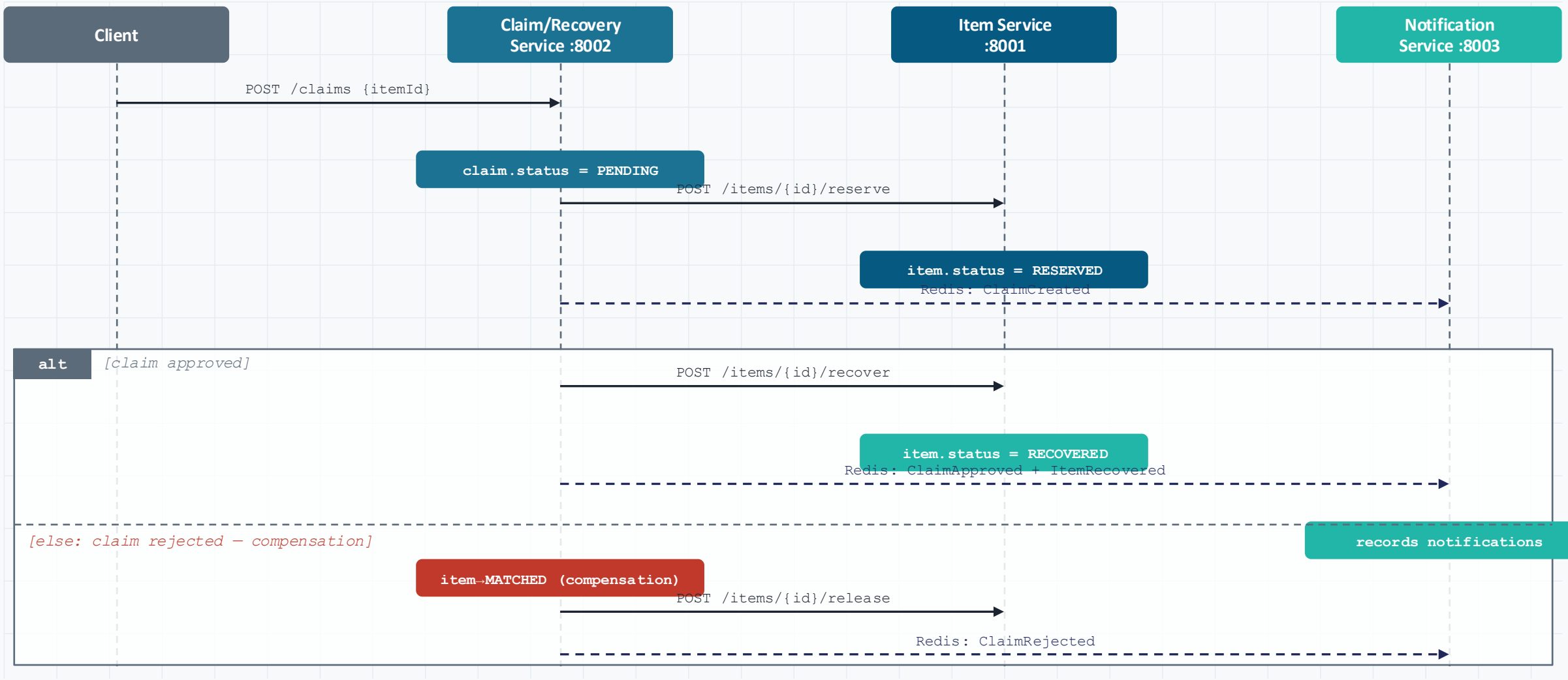
Trade-offs

- More inter-service communication
- Eventual consistency
- Saga coordination complexity

Happy Path — Claim & Recovery Flow

Compensation (rejection) path: Claim/Recovery calls Item Service to release the item back to MATCHED. See full Saga diagram →

Distributed Saga — Full Claim/Recovery Workflow



Compensation = new forward update releasing the item — never a database rollback | Orchestrated by Claim/Recovery Service | Mirrored in AWS Step Functions

CORE TECHNIQUES

Communication & Event-Driven Architecture

Synchronous — REST

Used when the client needs an immediate response

- `POST /register · POST /login` (Item Service)
- `POST /items` (Item Service)
- `POST /claims` (Claim/Recovery)
- `PUT /claims/{id}/approve|reject` (Claim/Recovery)

Asynchronous — Redis pub/sub

Channel: lostify:events | producer: Item Svc + Claim/Recovery | consumer: Notification Svc

Why Redis?

- Lightweight, zero-config pub/sub
- No external broker needed at this prototype scale

Idempotency

- Redis can redeliver a message
- Notification Service stores processed eventId values to prevent duplicates

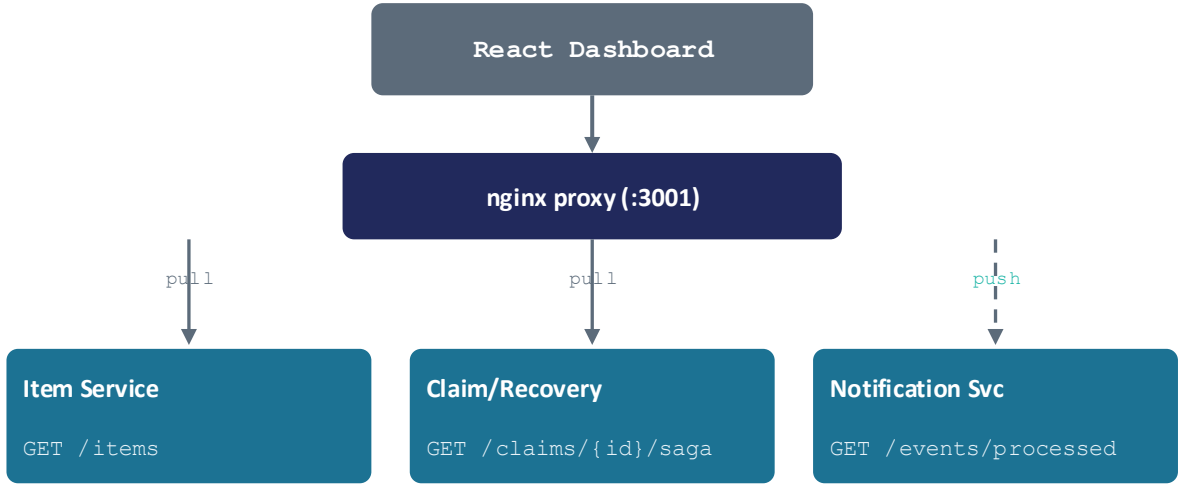
Event Schema

```
{ eventId, eventType,
  timestamp, payload }
```

CORE TECHNIQUES

Data Synchronization & Cross-Service Queries

API Composition — React Dashboard



Combined response — partial results shown if one service is slow/unavailable

pull: Dashboard fetches current state from Item + Claim/Recovery on page refresh **push:** Notification Service keeps its own log by listening to Redis events

Eventual Consistency

The notification log updates after the Redis event arrives — not inside the same transaction as item or claim creation.

Graceful Degradation

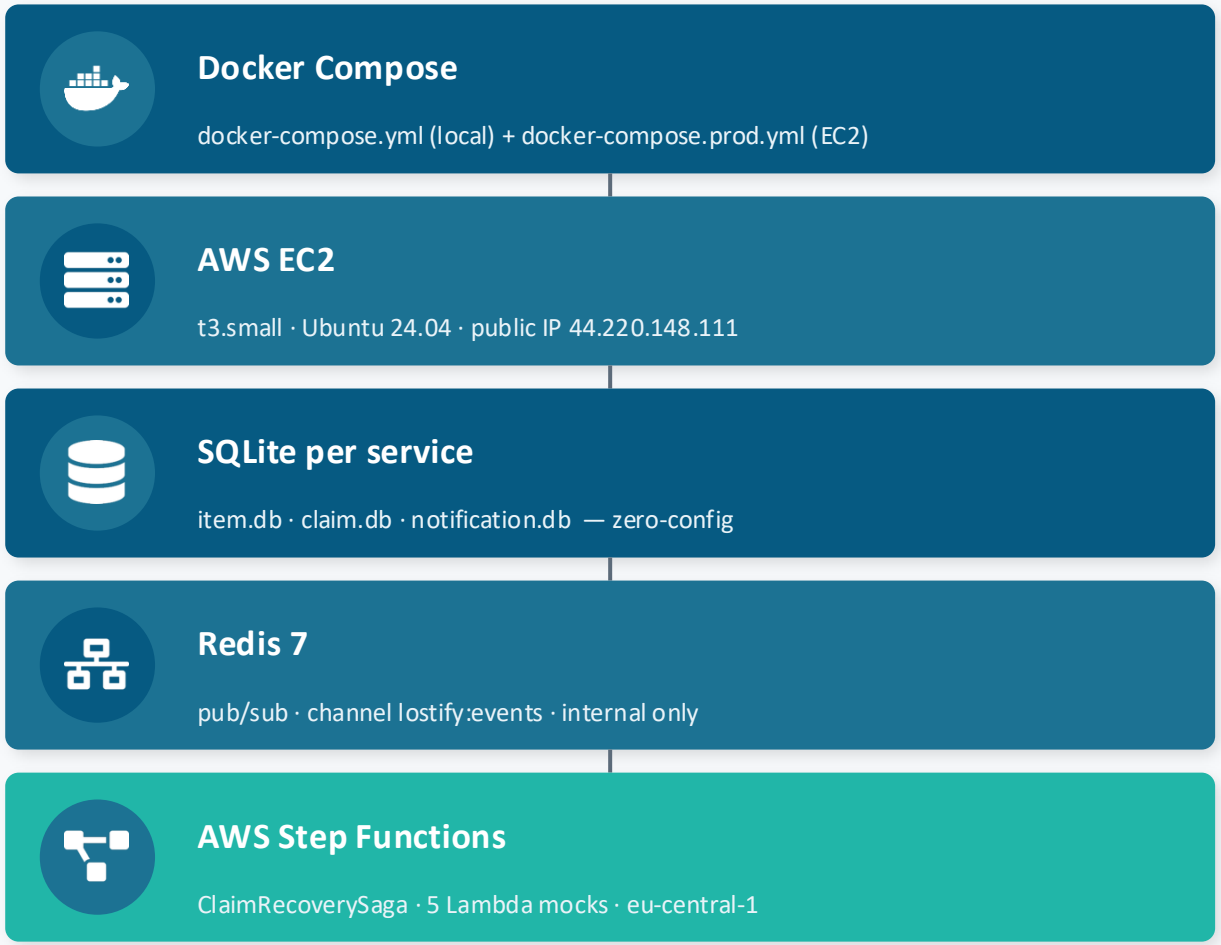
If Notification Service is unavailable, the dashboard still shows item and claim data instead of failing the whole page.

Future: CQRS Read Model

A dedicated read-optimised Dashboard Service could pre-build the combined view for faster loading.

CORE TECHNIQUES

Deployment & Cloud Infrastructure



GitHub Actions CI/CD

`ci.yml:` unit tests → integration tests → Docker build

`deploy.yml:` rsync → EC2 → docker compose up → health check

`Lead time:` ~3–5 min commit-to-production

Why this setup?

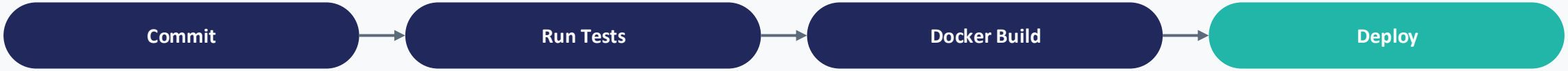
- Docker: each service deploys independently
- SQLite: zero-config; PostgreSQL on RDS is documented upgrade path
- Redis: lightweight pub/sub, no external broker needed

Portability Note

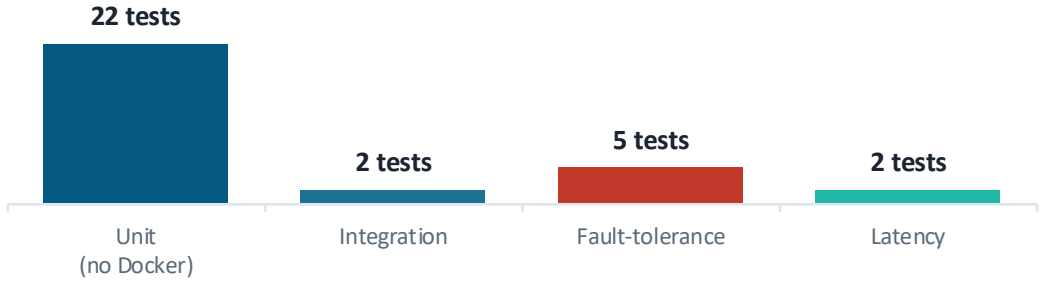
- SQLite → PostgreSQL upgrade path documented
- Event envelope in shared/events/ allows swapping Redis for another broker
- Step Functions Lambdas are mock handlers — local saga.py drives live demo

EVALUATION

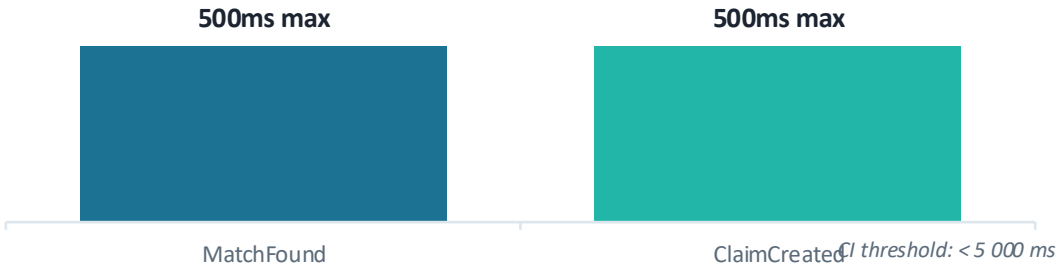
Automation & Evaluation Results



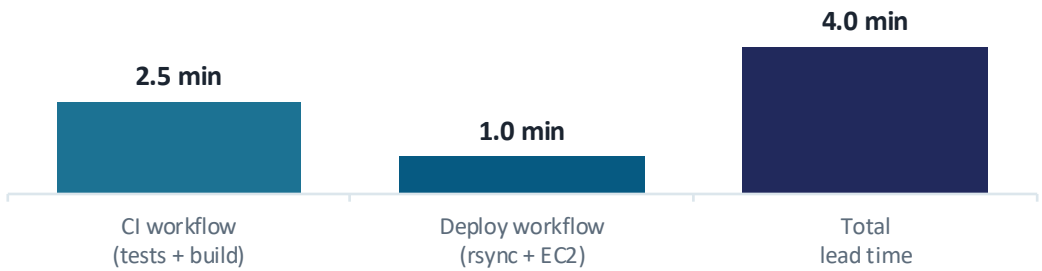
Automated Test Suite — 31 Tests



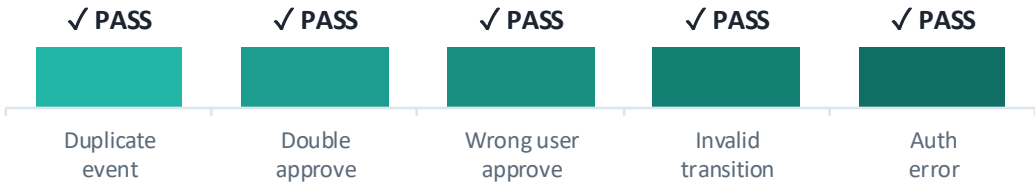
Async Event Latency — Observed Max (ms)



CI/CD Pipeline Duration (minutes)



Fault Tolerance — All Scenarios PASS



Six-part methodology: `Unit · Integration · Fault-tolerance · Event-latency · Demo scripts · Lead-time` | 3 / 3 services live on EC2

DISCUSSION

Threats to Validity

Prototype Scale

Tests use controlled descriptions; real user data may reduce matching accuracy.

Local vs. Cloud Env.

Docker Compose latency differs from EC2 network behavior and managed-service quirks.

Simplified Matching

Keyword overlap is transparent and easy to test, but not production-grade accuracy.

Limited Failure Injection

Not all combinations of distributed partial failure are covered in the test suite.

SQLite in MVP

SQLite concurrency profile differs from PostgreSQL on RDS — the documented upgrade path for production.

DISCUSSION

Challenges, Limitations & Future Work

Challenges

- Deciding the service split — auth embedded in Item Service per course guidance
- Designing stable Redis event payloads (eventId, eventType, timestamp, payload)
- Saga coordination: Claim/Recovery must call Item Service atomically with clear failure handling
- Coordinating API contracts across team members
- Measuring async latency alongside synchronous response times

Limitations

- No claim cancellation endpoint — rejection path is the only compensation in MVP
- Console-log notifications only — no email delivery (no AWS SES)
- SQLite instead of PostgreSQL (zero-config trade-off)
- Step Functions Lambdas are mock handlers — live Saga runs locally via saga.py
- No image storage — no AWS S3

Future Work

- PostgreSQL on AWS RDS (documented upgrade path)
- Email delivery via AWS SES
- Image storage on AWS S3
- Claim cancellation endpoint
- Better matching beyond keyword overlap
- Live Lambda handlers calling EC2 services

Thank You

Questions?

Zain Afzal · Babar Ali · Muhammad Hamza Azeem — Team 04

github.com/Babarali2k21/Lostify | 44.220.148.111:3001